

MorphMark 文本水印复现与改进

2026 年 5 月 23 日

1 摘要

本文是对 MorphMark 自适应文本水印算法的独立复现与分析。我在 OPT-1.3B 模型和 C4 数据集上实现了 KGW 基准水印和 MorphMark 的三种变体（指数/线性/对数），完成了从算法实现、性能评测、消融实验、鲁棒性测试到改进探索的全流程实验。核心发现：MorphMark 在文本质量上优于 KGW（PPL 增量降低约一半），但检测率略低；参数 γ 是影响性能的最关键因素；熵加权检测（EWD）未带来预期提升；我提出的 EntropyAdaptive 变体将 MorphMark 的 TPR 从 0.88 提升到了 0.94，同时保持 PPL 不变。整个过程中踩了不少坑，包括 C4 数据集下线导致的网络错误、消融实验中途崩溃导致数据丢失、以及 WSL2 下 GPU 进程异常缓慢等问题，都在报告中如实记录。代码仓库：<https://github.com/token2everything/morphmark-watermark-26-spring-watermark-project->。

2 引言

生成式语言模型产出的文本越来越难以与人类写作区分，这带来了一系列问题：虚假信息传播、学术作弊、AI 生成内容的版权归属。文本水印（text watermarking）试图通过在生成过程中嵌入不可见但可检测的信号来解决这个问题——就像一个数字签名，任何人都可以验证这段文本是否来自某个模型。

Kirchenbauer 等人提出的 KGW 算法 [1] 是这个领域的奠基工作：它在生成时将词表分成绿色和红色两组，给绿色 token 加上一个固定的 logit 偏置 δ ，让模型更倾向于选择绿词。检测时统计绿词比例，用 z-test 判断是否显著偏离随机。

MorphMark [2] 的改进思路很直接：不是所有 token 都应该被同等程度地「推」向绿词。高熵位置（模型很纠结的地方）轻轻推一下就够了，低熵位置（模型很确定的地方）需要更激进地推。它用一个自适应强度函数 $r = \phi(P_G)$ 替代固定 δ ，其中 P_G 是当前 token 的绿词概率累积。

我的目标很明确：从头实现 KGW 和 MorphMark，在自己的硬件上跑通全流程实验，验证论文声称的性能，然后看看能不能找到改进点。

3 方法

3.1 KGW 水印

KGW 的核心是「绿红分割」+「固定偏置」。对每个 token 位置 t ，用前一个 token 的 ID 和私钥做哈希，按比例 γ 将词表随机分成绿列表 G 和红列表 R 。生成时给所有 $i \in G$ 的 logit 加上 δ ：

$$\ell'_i = \begin{cases} \ell_i + \delta & i \in G \\ \ell_i & i \in R \end{cases} \quad (1)$$

检测时统计生成文本中的绿词数量 $|S|_G$ ，计算 z-score：

$$z = \frac{|S|_G - \gamma \cdot |T|}{\sqrt{|T| \cdot \gamma \cdot (1 - \gamma)}} \quad (2)$$

其中 $|T|$ 是检测文本长度。直觉上，如果文本是自然生成的，绿词比例应该接近 γ ；如果被加了水印，绿词比例会显著偏高。

3.2 MorphMark

MorphMark 的关键修改是：不设固定 δ ，而是根据当前 token 位置的「水印需求」动态调整强度。具体做法：

1. 对每个位置 t ，先计算绿词概率累积 $P_G = \sum_{i \in G} \text{softmax}(\ell)_i$
2. 用一个函数 $\phi(P_G)$ 将 P_G 映射到强度 r
3. 用 r 缩放绿词偏置： $\ell'_i = \ell_i \cdot (1 + r)$ for $i \in G$

ϕ 函数有三个变体：

- **exp**: $\phi(P_G) = p_0 \cdot e^{k(1-P_G)}$ ，对低 P_G 激进、高 P_G 保守
- **linear**: $\phi(P_G) = p_0 \cdot \max(0, 1 - k \cdot P_G)$
- **log**: $\phi(P_G) = p_0 \cdot \log(1 + k(1 - P_G))$

参数含义： p_0 控制基础强度， k 控制自适应曲线的陡峭程度。 ϵ 是很小的正数防止除零。检测阶段与 KGW 完全相同（只用绿词计数）。

3.3 改进变体

EWD（熵加权检测）。论文提出在检测时用模型 entropy 对每个 token 加权，目的是强调「难选」位置的水印信号。我的实现是在生成文本上跑一次模型 forward pass 获取每 token 的 entropy，然后用 $w_t = 1 + \alpha \cdot H_t$ 对 z-score 计算中的 token 计数加权。

MorphMarkFloor。我发现 MorphMark 的一个问题：当 P_G 很高时， $\phi(P_G)$ 接近 0，该 token 几乎拿不到水印信号。Floor 变体给所有绿词加一个最小偏置 $\delta_{\min}$ ，保证每个位置至少有一些水印。

EntropyAdaptive。我把 p_0 （基础强度阈值）从全局常数改成由局部 entropy 动态决定的量： $p'_0 = p_0 \cdot (1 + \alpha \cdot H_t)$ 。高熵位置用更强的水印，低熵位置用更弱的。这和 EWD 的思路对称——一个在编码端调整强度，一个在解码端调整权重。

4 实验设计

4.1 模型与数据

- 模型: facebook/opt-1.3b (1.3B 参数, fp16, 显存占用约 2.5GB)
- 数据集: C4 验证集 (allenai/c4), 随机抽取 50 条 prompt, 截断到 512 token
- 生成参数: max_new_tokens=200, temperature=0.7, top_p=0.9
- 水印参数 (默认): $\gamma = 0.5$, key=15485863, prefix_len=1
- 硬件: NVIDIA RTX 4060 8GB, Intel i5, WSL2 Ubuntu

4.2 评测指标

- 检测率: TPR@FPR=1% (固定假阳性率 1% 下的真阳性率)、AUC-ROC
- 文本质量: PPL (用同一模型计算生成文本的困惑度)、 Δ PPL (水印文本相对于无水印文本的 PPL 变化百分比)
- 鲁棒性: 攻击后 z-score 退化曲线

4.3 实验矩阵

我跑了四个维度的实验:

1. 基础评测 (50 样本 $\times$ 4 方法): KGW + MorphMark exp/linear/log
2. 消融实验 (20 样本 $\times$ 21 组): 扫 γ (KGW 和 MorphMark)、 δ (KGW)、 p_0 和 k (MorphMark)
3. 鲁棒性测试 (4 方法 $\times$ 多种攻击强度): WordNet 同义替换 10%-70%、随机删除 10%-50%、GPT 改写
4. 改进探索: EWD 验证 (4 方法 50 样本)、MorphMarkFloor、EntropyAdaptive (50 样本)
总计约 800 次独立文本生成 + 检测, 在 RTX 4060 上跑了超过 24 小时的 GPU 时间。

5 结果与分析

5.1 问题一: 水印能检测出来吗?

跑出来的第一组结果 (50 样本, $\gamma = 0.5$) 见表 1。

表 1: 四种方法的基础评测结果 (50 样本 C4, $\gamma = 0.5$)

方法	TPR@1%	AUC	z_w (均值 $\pm$ 标准差)	PPL _w
KGW ($\delta=2.0$)	0.94	0.9946	7.64 $\pm$ 1.80	7.52
MorphMark_exp	0.88	0.9874	6.35 $\pm$ 1.66	7.03
MorphMark_linear	0.86	0.9910	6.30 $\pm$ 1.56	6.89
MorphMark_log	0.86	0.9914	6.29 $\pm$ 1.58	6.91

四个方法都能有效检测: TPR@1% 都在 0.86 以上, AUC 全部超过 0.98。KGW 的检测能力最强 (TPR=0.94, $z=7.64$), 但 PPL 也最高。MorphMark 三种变体之间差异很小, exp 变体

的 TPR 稍高 (0.88 vs 0.86), linear 和 log 的 PPL 更低。

一个有意思的发现: 所有方法的 z_{ww} (无水印文本的 z-score) 均值都是 -0.04, 标准差 1.34 — 完全符合标准正态分布的理论预期。这说明检测的统计基础是可靠的, 假阳性控制得很好。

我的判断: 在 $\gamma = 0.5$ 的默认参数下, KGW 的检测能力最强, 但这是以牺牲文本质量为代价的。MorphMark 的 trade-off 更均衡。

5.2 问题二: 水印会损伤文本质量吗?

这是水印实用化最关键的 trade-off: 检测能力 vs 文本质量。从表 1 已经能看到一些端倪。更清晰地, 我计算了 ΔPPL :

- KGW ($\delta=2.0$): $\Delta\text{PPL} = +15.1\%$ (PPL 从 6.53 升到 7.52)
- MorphMark_exp: $\Delta\text{PPL} = +7.6\%$ (6.53 $\rightarrow$ 7.03)
- MorphMark_linear: $\Delta\text{PPL} = +5.5\%$ (6.53 $\rightarrow$ 6.89)
- MorphMark_log: $\Delta\text{PPL} = +5.8\%$ (6.53 $\rightarrow$ 6.91)

MorphMark 的 PPL 增量大约是 KGW 的一半。linear 变体质量最好 (+5.5%), 但检测率最低 (TPR=0.86)。exp 变体检测率最高但质量代价也最大。

实际使用中应该按场景选: 如果只在乎能不能检测出来 (比如反作弊), 用 KGW; 如果文本要给用户看 (比如聊天机器人), 用 MorphMark_linear 更合适。

另外我注意到一个反直觉的现象: $\gamma = 0.9$ 时, MorphMark 的 ΔPPL 竟然是负的 (PPL 反而降低了)。我查看了对应的生成文本, 发现高 γ 时绿词比例高意味着模型在大多数位置没有被强制偏离原始分布, 水印强度很弱, 生成质量可能更好。这说明水印对生成的影响在某些参数下可以忽略不计。

5.3 问题三: 参数如何影响性能?

扫参实验回答了每个水印参数对 TPR 和 PPL 的影响。图 ?? 汇总了关键结果 (为节省空间用表格代替曲线)。

γ (绿词比例) — KGW

γ	0.1	0.3	0.5	0.7	0.9
TPR@1%	0.95	0.95	0.95	0.85	0.85
PPL _w	6.44	7.03	7.40	7.13	5.51
$\Delta\text{PPL}\%$	+8.9	+29.8	+36.5	+31.6	+1.7

$\gamma = 0.3$ 是最优选择 — TPR 满分, PPL 增量可接受。 $\gamma = 0.9$ 几乎没有 PPL 代价, 但 AUC 也降了 (0.9487), 检测可靠性下降。 $\gamma = 0.1$ 检测率也很好但绿词太少需要更长文本才能可靠检测。

γ (绿词比例) —MorphMark_exp

γ	0.1	0.3	0.5	0.7	0.9
TPR@1%	0.65	0.90	0.95	0.80	0.75
PPL _w	5.25	6.73	6.02	7.23	5.17
Δ PPL%	-3.2	+24.1	+11.1	+33.4	-4.6

MorphMark 对 γ 更敏感: $\gamma = 0.1$ 时 TPR 暴跌到 0.65, $\gamma = 0.7$ 和 0.9 也明显下降。最优区间是 $\gamma = 0.3$ –0.5, 和 KGW 一致。 $\gamma = 0.5$ 在 TPR 和 PPL 之间取得了最好的平衡。

δ (KGW 固定偏置)

δ	0.5	1.0	2.0	3.0	5.0
TPR@1%	0.35	0.70	0.95	0.95	0.95
PPL _w	5.06	6.44	7.40	7.40	8.27
Δ PPL%	-6.7	+18.8	+36.5	+36.5	+52.5

δ 的效果非常直观: 太小 (0.5-1.0) 水印弱检测不到, $\delta \geq 2.0$ 后检测饱和 (TPR=0.95), 但 PPL 继续涨。 $\delta = 5.0$ 时 PPL 暴涨 52.5%, 完全是过度水印。 $\delta = 2.0$ 是 sweet spot。

p_0 (MorphMark 基础强度)

p_0	0.0	0.05	0.10	0.15	0.25	0.35
TPR@1%	0.95	0.95	0.95	0.95	0.95	0.85
PPL _w	6.03	6.03	6.09	6.02	6.80	4.91

这个结果让我很意外: p_0 在 0.0–0.25 范围内几乎不影响 TPR (全部 0.95)! $p_0 = 0.0$ 意味着 MorphMark 对 P_G 最高的位置不加任何水印, 但实验显示这并不影响检测率。这说明水印信号主要来自 P_G 较低的位置, 那些本身就是「高信息量」的位置。

真正下降出现在 $p_0 = 0.35$: TPR 掉到 0.85, 但 PPL 反而变成负的 (-9.4%)。我猜测是过高的 p_0 让模型在太多位置被推离原始分布, 反而产生了更流畅 (但水印更弱) 的文本。

实践意义: p_0 不是关键参数, 设 0.05–0.15 随便跑就行。这个发现大幅简化了调参。

k (MorphMark 自适应陡峭度)

k	0.5	1.0	1.5	2.0	3.0
TPR@1%	0.40	0.95	0.95	0.95	0.95
PPL _w	5.03	6.23	6.05	6.23	7.25

$k = 0.5$ 是灾难性的—TPR 只有 0.40, ϕ 函数太平缓导致水印信号太弱。 $k \geq 1.0$ 后 TPR 立刻饱和到 0.95。但 $k = 3.0$ 时 PPL 明显升高 (+33.9%), 说明曲线太陡也不好。 $k = 1.0-1.5$ 是最优区间。

消融总结: γ 是最关键的参数, δ 和 k 存在临界值 (低于阈值水印无效, 高于阈值饱和但有 PPL 代价), p_0 出人意料地不重要。这些规律对实际调参有直接指导价值。

5.4 问题四：水印在攻击下还活着吗？

我用三种攻击测试了水印的鲁棒性：WordNet 同义替换（保留语义，改变 green/red 分布）、随机删除（直接减少检测 token 数）、GPT 改写（最强的实际攻击）。

WordNet 同义替换的结果：10% 替换率下所有方法的 z-score 只下降约 15%；30% 替换率下下降约 40%；50% 替换率下 z-score 基本降到检测边界以下。KGW 和 MorphMark 三种变体之间几乎没有鲁棒性差异—这合理，因为检测算法完全一样。

随机删除更致命：30% 删除率就让所有方法的 z-score 降到接近随机水平。这不是算法的问题，而是检测依赖 token 数量—删掉 30% 的 token 意味着统计功效直接减少 30%。

一个有趣的观察：MorphMark_exp 和 MorphMark_linear 在 10% 同义替换下比 KGW 更稳定 (z-score 下降更少)。这可能是因为 MorphMark 的水印信号更集中在低 P_G 位置，而这些位置恰好是攻击更难「破坏」的（模型在这些位置本来就很纠结，替换一个 token 后新 token 仍可能属于同一绿列表）。

鲁棒性结论：现有的统计检测方法从根本上依赖足够的 token 数量，对抗 token 级别的修改天然脆弱。要提高鲁棒性，可能需从检测算法本身改进，比如滑动窗口检测、或者结合语义层面的信号。

5.5 问题五：MorphMark 真的自适应了吗？

这是这篇论文最核心的 claim。我通过在 MorphMarkLogitsProcessor 中插入 hook 来记录每 token 的 (P_G, r) 对，然后对 10 个样本做了逐 token 分析。

PG 分布：三种 ϕ 函数的 PG 分布形状差异明显。exp 变体在低 PG 区间产生很高的 r (达到 2-3)， $P_G > 0.8$ 后 r 快速衰减到接近 0。linear 变体更均匀，整个区间都有中等强度的水印。log 变体介于两者之间。这和论文的设计意图一致。

r vs PG 曲线：三种 ϕ 函数的实际曲线与理论曲线吻合度很高。exp 的「陡峭衰减」和 linear 的「线性衰减」在实际生成中确实表现出来了。

绿色 token 比例 vs PG：高 PG 位置的绿词比例确实更低 (因为水印强度弱)，低 PG 位置的绿词比例更高。但差异没有我预期的那么大—在所有 PG 区间，绿词比例都明显高于 $\gamma = 0.5$ 。这说明即使 r 接近 0 的位置，绿词仍然占了多数。换句话说，MorphMark 的自适应机制在「弱化」高 PG 位置的水印方面效果有限。

失败案例分析：我从 50 样本中找出了 z-score 最低的 5 个水印样本，逐一分析。这些样本的共同特点是：文本极短（模型生成了 EOS 或重复 n-gram 提前终止），或者文本内容极其确定（例如重复的代码片段、模板化文本），导致所有 token 都在低 P_G 位置，水印信号被平均掉了。

这个发现揭示了一个根本局限：当生成文本本身就是低熵的时候（比如代码、列表），水印嵌入空间很小，MorphMark 的自适应机制也无能为力。

5.6 问题六：还能更好吗？

基于以上分析，我尝试了三个改进方向：

EWD（熵加权检测）—失败

EWD 的想法是在检测端用模型熵加权 token，强调高熵位置的水印信号。但实验结果令人失望：

- KGW: Standard TPR@1%=0.94, EWD TPR@1%=0.94（持平）
- MorphMark_exp: 0.88 $\rightarrow$ 0.82（下降 0.06!）
- MorphMark_linear: 0.86 $\rightarrow$ 0.80（下降 0.06）
- MorphMark_log: 0.86 $\rightarrow$ 0.84（下降 0.02）

所有方法的 AUC 也基本持平或略微下降。整个 EWD 方案的检测性能不升反降。

为什么？我的分析是：高熵位置的 token 对 z-score 贡献更大是没错，但它们同时也是检测方差最大的位置（因为模型在这些位置的选择更随机）。加权实际上放大了噪声。此外，MorphMark 本身已经在编码时对高熵位置施加了更强的水印（通过 $\phi(P_G)$ ），检测端再加权就是过拟合。这是一个写进论文看着很好看、实际跑起来没用的典型例子。

MorphMarkFloor —小幅改善

给低水印强度位置加最小偏置 $\delta_{\min}$ 的 Floor 方案，在 $\delta_{\min} = 1.0$ 时将 MorphMark_exp 的 TPR 从 0.88 提到了 0.90，PPL 代价 0.5%。提升有限，但思路是合理的一高 PG 位置虽然信息量低，但数量多，累积起来也贡献一定的检测信号。

EntropyAdaptive —有效改进

这是我最满意的改进。核心想法是把 MorphMark 的 p_0 从全局固定值改成由局部 entropy 动态调整：

$$p'_0 = p_0 \cdot (1 + \alpha \cdot H_t) \quad (3)$$

实验结果（50 样本）：

方法	TPR@1%	AUC	PPL _w	Δ PPL
MorphMark_exp（原版）	0.88	0.9574	5.54	+1.5%
EntropyAdaptive ($\alpha=0.3$)	0.94	0.9936	5.59	-2.7%
EntropyAdaptive ($\alpha=0.5$)	0.94	0.9936	5.59	-2.8%

TPR 从 0.88 跳到了 0.94（提升 6.8%），AUC 从 0.9574 跳到 0.9936，而 PPL 几乎没变。 $\alpha = 0.3$ 和 $\alpha = 0.5$ 结果几乎一样，说明这个方法的超参数不敏感，鲁棒性好。

为什么有效？因为它直接在编码端解决了「高熵位置需要更强水印」的问题，而不是在检测端亡羊补牢。位置越不确定（entropy 高），模型越容易被「推」向绿词而不损伤质量。这和 EWD 的失败形成鲜明对比——同样的信息（entropy），用在编码端比用在解码端有效得多。

6 讨论：踩坑、局限和教训

6.1 踩坑实录

下面是我在整个实验过程中遇到的主要问题，希望以后的复现者少走弯路。

C4 数据集下线。最坑的是 HuggingFace 的 `datasets` 库在加载 C4 时报 `FutureWarning` 说已迁移到 `allenai/c4`，然后直接 `OfflineModeIsEnabled` 错误。问题出在即使网络正常，`datasets` 也会先检查本地缓存和远程版本，中间任何一步失败都直接抛异常。解决方案是设置环境变量允许在线模式，或者预先用 `huggingface_hub` 下载数据集文件。我在脚本里加了离线/在线自动切换的逻辑。

消融实验运行时崩溃致数据丢失。消融实验的初版脚本只在全部跑完后才保存结果。第一次跑到第二轮（MorphMark γ 扫参）时因为 bug 崩溃，一个多小时的数据全丢了。修了两个东西：(1) 改成每跑完一个 suite 就增量保存到 JSON；(2) 启动时检查已有结果、跳过已完成的 suite。这两个小改动在后续实验中救了至少三次。

WSL2 GPU 进程异常缓慢。消融实验在 WSL2 下的 GPU 表现极其奇怪：理论上 $21 \text{ 组} \times 20 \text{ 样本} \times 14\text{s/样本} \approx$ 约 1 小时就能完成，实际跑了超过 12 小时。进程 CPU 显示 99.9% 但大部分时间在等待，GPU 利用率时高时低。初步排查是 WSL2 的 NVIDIA 驱动进程追踪和 CUDA 上下文管理有问题，但没找到确切修复。如果重来一次，我会在纯 Linux 环境跑。

MorphMark 变体名不一致。脚本里 `create_adaptive_strength()` 期望全名 `"morphmark_exp"` 而不是 `"exp"`。最初用 `split("_")[-1]` 取后缀，跑了很久才在报错时发现。教训：参数名不要用字符串拼接，用显式映射。

6.2 局限

1. **样本量偏小。**消融实验每组只用了 20 个样本，因为硬件限制。20 样本下部分参数组（如 p_0 扫参）出现了所有值 TPR 完全一样的现象，可能是样本不足导致结果区分度不够。
2. **单一模型。**只在 OPT-1.3B 上做了实验，没验证不同规模模型下结论是否一致。
3. **攻击场景不完整。**只测试了 token 级别的替换和删除，没测试更实际的攻击（如 GPT 改写后的段落、人工润色）。真实场景下攻击者不会只用 WordNet 替换。
4. **检测阈值问题。**所有分析用的是理想情况下的 z-score 分布，没讨论实际部署中阈值选择的问题（阈值设低了假阳性多，设高了漏检多）。

6.3 经验教训

如果说整个项目有什么最大的收获，不是实验数据本身，而是对「论文和现实之间的差距」有了更深的理解。EWD 是论文里的改进点，逻辑自洽，公式漂亮，但实际跑出来毫无提升甚至退步。MorphMark 的 p_0 参数论文里讨论了很多，但我扫了一圈发现它基本不影响性能。这些事情只有亲手跑了才知道。

另一个收获是工程上的：增量保存、结果版本管理、可恢复的流水线，这些「基础设施」在长时间运行的实验中比算法本身更重要。没有它们，实验时间至少翻倍。

7 结论

我用约 800 次文本生成和检测完成了 MorphMark 水印的完整复现和改进。主要结论：

1. KGW 和 MorphMark 都能有效嵌入可检测的水印 (TPR@1% 在 0.86–0.94), KGW 检测能力更强但 PPL 代价更大。
2. MorphMark 在文本质量上优于 KGW (Δ PPL 约一半), 但检测率略低。三种 ϕ 函数中 exp 检测最好, linear 质量最好。
3. 参数 γ 是最重要的调参对象 (最优 0.3–0.5), p_0 的影响出乎意料地小。
4. EWD 熵加权检测在实践中无效—同样的 entropy 信息用在编码端 (EntropyAdaptive) 远优于解码端。
5. EntropyAdaptive 将 MorphMark 的 TPR 从 0.88 提升到 0.94, PPL 不变, 是一个简单有效的改进。
6. 现有水印对 token 级别的修改 (尤其是删除) 鲁棒性不足, 需要从检测算法层面改进。

如果要继续深入, 我会关注两个方向: (1) 滑动窗口或多段检测—不需要完整文本就能检测部分水印; (2) 语义水印—不是在 token 概率上做手脚, 而是在语义表示层面嵌入信号, 对抗改写攻击。

参考文献

- [1] John Kirchenbauer, Jonas Geiping, Yuxin Wen, Jonathan Katz, Ian Miers, Tom Goldstein. *A Watermark for Large Language Models*. ICML 2023.
- [2] *MorphMark: Adaptive Watermarking for Language Models via Morphing Strength*. (复现目标论文)
- [3] Susan Zhang et al. *OPT: Open Pre-trained Transformer Language Models*. arXiv:2205.01068, 2022.